

ML_lab1_preprocessing

August 24, 2026

0.0.1 Perform various data preprocessing techniques like handling missing data and feature scaling.

step 1: Start by importing the necessary Python libraries for data preprocessing.

```
[18]: import pandas as pd
import numpy as np
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, MinMaxScaler
```

Step 2: Load the placement dataset into a Pandas Dataframe.

```
[19]: df=pd.read_csv("ML/data.csv")
df.head(10)
```

```
[19]:
```

	sl_no	gender	hsc_p	hsc_s	degree_p	degree_t	etest_p	specialisation \
0	1	M	91.00	Commerce	58.00	Sci&Tech	55.00	Mkt&HR
1	2	M	78.33	Science	77.48	Sci&Tech	86.50	Mkt&Fin
2	3	M	NaN	Arts	64.00	Comm&Mgmt	75.00	Mkt&Fin
3	4	M	52.00	Science	NaN	Sci&Tech	66.00	Mkt&HR
4	5	M	73.60	Commerce	73.30	Comm&Mgmt	96.80	Mkt&Fin
5	6	M	49.80	Science	67.25	Sci&Tech	55.00	Mkt&Fin
6	7	F	49.20	Commerce	79.00	Comm&Mgmt	74.28	Mkt&Fin
7	8	M	64.00	Science	66.00	Sci&Tech	67.00	Mkt&Fin
8	9	M	79.00	Commerce	72.00	Comm&Mgmt	91.34	Mkt&Fin
9	10	M	70.00	Commerce	61.00	Comm&Mgmt	54.00	Mkt&Fin

	mba_p	salary
0	58.80	270000.0
1	66.28	200000.0
2	57.80	250000.0
3	59.43	NaN
4	55.50	425000.0
5	51.58	NaN
6	53.29	NaN
7	62.14	252000.0
8	61.29	231000.0
9	52.21	NaN

Step 3:Take a quick look at the data to understand its structure and identify any missing values or anomalies.

```
[20]: df.shape
      df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 215 entries, 0 to 214
Data columns (total 10 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   sl_no                 215 non-null    int64
 1   gender                215 non-null    object
 2   hsc_p                 210 non-null    float64
 3   hsc_s                 215 non-null    object
 4   degree_p              213 non-null    float64
 5   degree_t              215 non-null    object
 6   etest_p               211 non-null    float64
 7   specialisation        215 non-null    object
 8   mba_p                 214 non-null    float64
 9   salary                148 non-null    float64
dtypes: float64(5), int64(1), object(4)
memory usage: 16.9+ KB
```

The method `isnull()` checks each element in the **DataFrame** (or **Series**) to see if it is **NaN** (Not a Number) or **None** (missing value). It returns a **DataFrame** (or **Series**) of the same shape as the input, with Boolean values: `#### True: The value is null (NaN or None).`
`#### False: The value is not null.`

```
[21]: df.isnull().sum()
```

```
[21]: sl_no          0
      gender        0
      hsc_p         5
      hsc_s         0
      degree_p      2
      degree_t      0
      etest_p       4
      specialisation 0
      mba_p         1
      salary       67
      dtype: int64
```

Step 4: Handle Missing Data

Option 1: If the dataset is large and a significant percentage of the data is missing (and the feature is not important), you can remove rows with missing values.

```
[27]: df.dropna(subset=["salary"], inplace=True)
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 148 entries, 0 to 213
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   sl_no                 148 non-null   int64
1   gender                148 non-null   object
2   hsc_p                 148 non-null   float64
3   hsc_s                 148 non-null   object
4   degree_p              148 non-null   float64
5   degree_t              148 non-null   object
6   etest_p               148 non-null   float64
7   specialisation        148 non-null   object
8   mba_p                 148 non-null   float64
9   salary                148 non-null   float64
dtypes: float64(5), int64(1), object(4)
memory usage: 12.7+ KB
```

```
[28]: df.head()
```

```
[28]:   sl_no  gender  hsc_p  hsc_s  degree_p  degree_t  etest_p  \
0      1      M  91.00000  Commerce    58.00  Sci&Tech    55.0
1      2      M  78.33000   Science    77.48  Sci&Tech    86.5
2      3      M  70.00089     Arts    64.00  Comm&Mgmt    75.0
4      5      M  73.60000  Commerce    73.30  Comm&Mgmt    96.8
7      8      M  64.00000   Science    66.00  Sci&Tech    67.0

   specialisation  mba_p  salary
0      Mkt&HR    58.80  270000.0
1      Mkt&Fin    66.28  200000.0
2      Mkt&Fin    57.80  250000.0
4      Mkt&Fin    55.50  425000.0
7      Mkt&Fin    62.14  252000.0
```

Option 2: If removing data isn't ideal, you can impute (fill in) missing values using methods like mean, median, or most frequent.

```
[29]: hsc_p_mean = df['hsc_p'].mean()
etest_p_mean = df['etest_p'].mean()
degree_p_mean = df['degree_p'].mean()
mba_p_mean = df['mba_p'].mean()

df['hsc_p'].fillna(hsc_p_mean, inplace=True)
df['etest_p'].fillna(etest_p_mean, inplace=True)
df['degree_p'].fillna(degree_p_mean, inplace=True)
```

```
df['mba_p'].fillna(mba_p_mean, inplace=True)
```

C:\Users\admin\AppData\Local\Temp\ipykernel_16224\1882724610.py:6:
FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['hsc_p'].fillna(hsc_p_mean, inplace=True)
```

C:\Users\admin\AppData\Local\Temp\ipykernel_16224\1882724610.py:7:
FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['etest_p'].fillna(etest_p_mean, inplace=True)
```

C:\Users\admin\AppData\Local\Temp\ipykernel_16224\1882724610.py:8:
FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['degree_p'].fillna(degree_p_mean, inplace=True)
```

C:\Users\admin\AppData\Local\Temp\ipykernel_16224\1882724610.py:9:
FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['mba_p'].fillna(mba_p_mean, inplace=True)
```

Step 5: Feature Scaling ##### Option 1: This method scales the data to have a mean of 0 and a standard deviation of 1.

```
[38]: numeric_columns = ['hsc_p', 'degree_p', 'mba_p', 'salary']
      scaler = StandardScaler()
      df[numeric_columns] = scaler.fit_transform(df[numeric_columns])

      df.describe()
```

```
[38]:
```

	sl_no	hsc_p	degree_p	etest_p	mba_p \
count	148.000000	1.480000e+02	1.480000e+02	148.000000	1.480000e+02
mean	106.878378	-1.200241e-17	2.820567e-16	73.140890	-3.600723e-17
std	60.682502	1.003396e+00	1.003396e+00	13.701352	1.003396e+00
min	1.000000	-2.068715e+00	-1.960213e+00	50.000000	-1.739125e+00
25%	57.750000	-7.554605e-01	-5.745741e-01	60.000000	-8.196357e-01
50%	108.500000	-1.727508e-01	-1.126944e-01	72.000000	-5.701803e-02
75%	153.250000	5.664272e-01	5.681931e-01	85.000000	7.128464e-01
max	214.000000	2.988989e+00	3.428383e+00	98.000000	2.610652e+00

	salary
count	1.480000e+02
mean	1.800362e-16
std	1.003396e+00
min	-9.518389e-01
25%	-5.223834e-01
50%	-2.539736e-01
75%	1.218000e-01
max	6.993089e+00

Option 2: This method scales the data to a fixed range, usually between 0 and 1.

```
[39]: scaler = MinMaxScaler()
      df[numeric_columns] = scaler.fit_transform(df[numeric_columns])
      df.describe()
```

```
[39]:
```

	sl_no	hsc_p	degree_p	etest_p	mba_p	salary
count	148.000000	148.000000	148.000000	148.000000	148.000000	148.000000
mean	106.878378	0.409023	0.363771	73.140890	0.399819	0.119805
std	60.682502	0.198390	0.186207	13.701352	0.230677	0.126294
min	1.000000	0.000000	0.000000	50.000000	0.000000	0.000000
25%	57.750000	0.259654	0.257143	60.000000	0.211388	0.054054

50%	108.500000	0.374867	0.342857	72.000000	0.386711	0.087838
75%	153.250000	0.521016	0.469214	85.000000	0.563701	0.135135
max	214.000000	1.000000	1.000000	98.000000	1.000000	1.000000

Step 6: Separate the dataset into features (X) and target (y) variables. The target is usually the variable the model wants to predict.

```
[41]: X = df[['gender', 'hsc_p', 'hsc_s', 'degree_p', 'degree_t', 'etest_p',
        ↪ 'specialisation', 'mba_p',]]
y = df['salary']
```

0.0.2 Step 7: After preprocessing, save the cleaned and scaled dataset to a new CSV file

```
[45]: preprocessed_data = pd.concat([X, y], axis=1)

preprocessed_data.to_csv('preprocessed_data.csv', index=False)
```

These assessment tasks use `placement_assessment_dataset.csv` to evaluate students' decision-making in handling missing data and feature scaling.

Task 1: Handling Missing Data 1.1 High Missingness & Column Dropping

Dataset Context: `internship_score` contains 200 missing entries out of 250 total rows (80% missing).
Questions:

- Write Pandas code to verify the percentage of missing values per column.
- Explain why dropping `internship_score` is preferred over mean/median imputation.

```
[ ]:
```

1.2 Imputation Selection (Numerical & Categorical)

Dataset Context: `hsc_p` and `degree_p` have low missingness. `degree_t` has 10 missing categorical entries.

Questions:

- Write code using `SimpleImputer` to impute `hsc_p` and `degree_p` with their median.
- Impute missing `degree_t` values using the mode (most frequent category).

```
[ ]:
```

Task 2: Feature Scaling (Standardization vs. Normalization) 2.1 Scaling Behavior & Outliers

Dataset Context: `salary` ranges from \$111,000 to \$583,000 with significant right skew.

Questions:

- Compare the mathematical outputs of applying `StandardScaler` versus `MinMaxScaler` on `salary`.

ii) Which scaler prevents extremely high salaries from squashing the remaining data into a tiny range?

[]: